

INSTITUTO INFNET

VINICIUS REIS ZIMMERMANN

DESENVOLVIMENTO DE APLICAÇÕES JAVA COM SPRING BOOT

Relatório do projeto da disciplina

Curso: Engenharia de Software com Java

Relatório apresentado ao Instituto Infnet, no curso Engenharia de Software com Java, como parte da avaliação da disciplina Desenvolvimento de Aplicações Java com Spring Boot.

Repositório do projeto:

<https://github.com/Viniciusrz7/Trabalho-INFNET-Disciplina-Desenvolvimento-de-aplica-es-Java-com-Spring-Boot->

1 INTRODUÇÃO

Neste trabalho eu montei uma API REST para dar conta do dia a dia de uma lanchonete. Dá para cadastrar a lanchonete, os clientes, os itens do cardápio e os pedidos, e também consultar, buscar pelo nome, alterar e excluir tudo isso. Fiz o projeto em Java com Spring Boot e deixei o Swagger UI ligado, que é onde eu testei cada rota direto pelo navegador.

A intenção foi pegar o que a gente viu na disciplina e usar em um exemplo pequeno, mas que funcionasse de verdade do começo ao fim. Por isso separei a aplicação em camadas, gravei os dados em banco, coloquei validação em tudo que chega pelo corpo da requisição e me preocupei em devolver o código HTTP certo em cada situação.

Escrevi este relatório como um relato do que eu fiz e do que aprendi no caminho. Primeiro mostro os objetivos e as tecnologias, depois o modelo de domínio com as entidades e os relacionamentos, a validação, as camadas de controller, service e repository, o tratamento de erros e, no fim, os testes que rodei no Swagger. Coloquei bastante trecho de código junto das explicações, porque acho que fica bem mais fácil de entender vendo o código do que só lendo a descrição dele.

2 OBJETIVOS

O objetivo principal era construir uma API REST que representasse bem o contexto de uma lanchonete e que mostrasse na prática o uso do Spring Boot.

Junto com isso, eu queria dar conta de alguns pontos específicos: dividir o código entre controller, service e repository sem misturar as responsabilidades; mapear as entidades e os relacionamentos com JPA; usar herança no domínio; validar os campos com Bean Validation; fazer o CRUD completo, incluindo a alteração parcial; juntar o tratamento de erros em uma classe só; documentar tudo com OpenAPI; e, principalmente, provar que funciona rodando cada rota no Swagger e guardando a evidência.

3 TECNOLOGIAS E ESTRUTURA DO PROJETO

O projeto é Java com Spring Boot, construído pelo Maven. No pom.xml eu declarei o spring-boot-starter-webmvc, que cuida das rotas HTTP; o spring-boot-starter-data-jpa, que traz o Hibernate e o Spring Data; o banco H2, que roda em memória; o spring-boot-starter-validation, para o Bean Validation funcionar; e o springdoc-openapi-starter-webmvc-ui, que é quem gera a tela do Swagger.

A configuração do banco está no application.properties. Escolhi o H2 em memória porque ele sobe junto com a aplicação e some quando eu encerro, o que resolve para um trabalho da disciplina e me poupou de instalar banco separado na máquina.

Código 1 – Configuração do banco de dados no application.properties

```
spring.datasource.url=jdbc:h2:mem:vinicius_reis_zimmermann
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa

spring.jpa.hibernate.ddl-auto=create-drop
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console
spring.jpa.show-sql=true
```

O ddl-auto igual a create-drop faz o Hibernate criar as tabelas sozinho a partir das minhas entidades quando a aplicação sobe, e apagar tudo quando ela para. Já o show-sql joga no console o SQL que ele está gerando, e isso me ajudou demais: várias vezes eu só entendi o que estava acontecendo olhando a consulta que o Hibernate montou. O console do H2 fica no caminho /h2-console, caso queira espiar as tabelas com a aplicação ligada.

Organizei o código em pacotes seguindo a divisão em camadas, todos dentro de br.edu.infnet:

Quadro 1 – Organização dos pacotes do projeto

Pacote	O que guarda
model.domain	As entidades do sistema: Lanchonete, Cliente, ItemCardapio, Lanche, Bebida e Pedido.
repository	As interfaces que acessam o banco, uma para cada entidade concreta.
service	As regras da aplicação e as operações de CRUD.

controller	As classes que recebem as chamadas HTTP e devolvem as respostas.
exception	O tratamento global de erros, o corpo padrão do erro e as exceções que eu criei.

4 MODELO DE DOMÍNIO

O domínio ficou com seis classes. Lanchonete é o estabelecimento. Cliente é a pessoa cadastrada naquela lanchonete. ItemCardapio é uma classe abstrata com o que todo item do cardápio tem em comum, e dela saem Lanche e Bebida, cada uma com os dados que só fazem sentido para ela. Pedido é a compra em si, com o cliente, os itens escolhidos e a data e hora em que foi emitido.

4.1 Entidades e anotações de mapeamento

Toda entidade leva `@Entity`, que é o que avisa o JPA de que aquela classe vira tabela. A chave primária leva `@Id` junto com `@GeneratedValue` na estratégia `IDENTITY`, ou seja, quem gera o número é o próprio banco e eu não preciso me preocupar com isso. O `@Column` serve para ajustar a coluna, como tamanho e obrigatoriedade, e o `@Table` eu só usei quando quis que o nome da tabela ficasse diferente do nome da classe.

Código 2 – Trecho da entidade Lanchonete

```
@Entity
@Table(name = "Lanchonete")
public class Lanchonete implements Identificavel {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank(message = "O nome deve ser informado")
    @Size(max = 150, message = "O nome deve possuir no máximo 150 caracteres")
    @Column(nullable = false, length = 150)
    private String nome;

    @NotBlank(message = "O CNPJ é obrigatório")
    @CNPJ(message = "CNPJ inválido")
    @Column(nullable = false, unique = true, length = 18)
    private String cnpj;
}
```

Nesse trecho tem dois tipos de anotação que parecem fazer a mesma coisa, mas não fazem. O `@Column` conversa com o banco: o `nullable` igual a `false`

cria a coluna como obrigatória e o `unique` igual a `true` impede dois registros com o mesmo CNPJ. Já o `@NotBlank`, o `@Size` e o `@CNPJ` conversam com o `Bean Validation`, que confere o dado antes de chegar perto do banco e ainda devolve uma mensagem explicando o que está errado. Se eu tivesse deixado só a restrição do banco, o usuário da API receberia um erro feio de `constraint` em vez de uma frase dizendo qual campo precisa arrumar.

Cliente segue a mesma linha, com o CPF validado pelo `@CPF`, que confere os dígitos verificadores. Essas duas anotações, `@CPF` e `@CNPJ`, vêm do `Hibernate Validator` e já trazem prontas as regras dos documentos brasileiros, então não precisei escrever essa conta na mão.

4.2 Herança de ItemCardapio

Deixei `ItemCardapio` abstrata porque não existe item genérico na lanchonete: ou é lanche, ou é bebida. Ela guarda o que os dois têm em comum, que é nome, preço, disponibilidade e a lanchonete dona do item, e declara o método abstrato `descreverPreparo`. Cada subclasse implementa esse método do seu jeito, e é aí que aparece o polimorfismo: a chamada é a mesma, mas o texto que sai depende de o objeto ser um lanche ou uma bebida.

Código 3 – Herança e conversão em JSON da classe `ItemCardapio`

```
@Entity
@Table(name = "itemcardapios")
@Inheritance(strategy = InheritanceType.JOINED)
@JsonTypeInfo(use = JsonTypeInfo.Id.NAME, property = "tipo")
@JsonSubTypes({
    @JsonSubTypes.Type(value = Lanche.class, name = "lanche"),
    @JsonSubTypes.Type(value = Bebida.class, name = "bebida")
})
public abstract class ItemCardapio implements Identificavel {

    @NotNull(message = "O preço é obrigatório")
    @Positive(message = "O preço deve ser maior que zero")
    @Column(nullable = false, precision = 10, scale = 2)
    private BigDecimal preco;

    public abstract String descreverPreparo();
}
```

O `@Inheritance` com a estratégia `JOINED` é o que decide como essa herança vai parar no banco. Os campos comuns ficam na tabela `itemcardapios` e os campos específicos ficam nas tabelas de lanche e de bebida, ligadas pela

mesma chave. Gostei dessa estratégia porque não sobra coluna vazia, e mesmo assim a consulta geral do cardápio continua trazendo os dois tipos misturados na mesma lista.

Já o `@JsonTypeInfo` com o `@JsonSubTypes` resolve outro problema, do lado do JSON. Como a classe é abstrata, o Jackson não tem como adivinhar se deve criar um lanche ou uma bebida quando chega um cadastro. Por isso o JSON precisa mandar o campo tipo com o valor lanche ou bebida, que funciona como uma etiqueta e também aparece nas respostas. Descobri isso na marra: o primeiro cadastro que tentei pela rota geral de itens voltou erro, e só passou depois que incluí esse campo no corpo da requisição.

Uma escolha que fiz de propósito foi usar `BigDecimal` no preço em vez de `double`. Valor em dinheiro com ponto flutuante acaba dando aquelas diferenças de centavo em conta, e o `@Column` com `precision 10` e `scale 2` já reserva as duas casas dos centavos.

4.3 Relacionamentos entre as entidades

Os relacionamentos foram todos mapeados com anotações do JPA. O quadro abaixo resume como cada ligação ficou e o que ela quer dizer na prática da lanchonete.

Quadro 2 – Relacionamentos mapeados com JPA

Relacionamento	Anotação usada	O que significa
Lanchonete e ItemCardapio	<code>@OneToMany(mappedBy = "lanchonete")</code> de um lado e <code>@ManyToOne</code> com <code>@JoinColumn</code> do outro	Uma lanchonete tem vários itens no cardápio, e cada item pertence a uma lanchonete só.
Lanchonete e Cliente	<code>@OneToMany</code> e <code>@ManyToOne</code>	Uma lanchonete tem vários clientes cadastrados, e o cliente está ligado a uma lanchonete.
Lanchonete e Pedido	<code>@OneToMany</code> e <code>@ManyToOne</code>	A lanchonete guarda o histórico dos pedidos que passaram por ela.
Pedido e Cliente	<code>@ManyToOne</code> com <code>@JoinColumn(name = "cliente_id")</code>	Cada pedido é de um cliente, e o mesmo cliente pode fazer vários pedidos.
Pedido e ItemCardapio	<code>@ManyToMany</code> com <code>@JoinTable(name =</code>	Um pedido tem vários itens e o mesmo item aparece em vários pedidos. Por isso o banco

	"pedido_item")	cria a tabela do meio, chamada pedido_item.
--	----------------	---

O mappedBy é o que diz qual lado manda no relacionamento. Quem tem mappedBy só espelha a ligação; quem tem @JoinColumn é quem de fato grava a chave estrangeira na própria tabela. No muitos-para-muitos entre pedido e item nenhum dos dois consegue guardar isso sozinho, e aí entra o @JoinTable descrevendo a tabela intermediária.

Código 4 – Relacionamentos da entidade Pedido

```
@NotNull(message = "O cliente é obrigatório")
@ManyToOne
@JoinColumn(name = "cliente_id", nullable = false)
private Cliente cliente;

@NotEmpty(message = "O pedido deve possuir pelo menos um item")
@ManyToMany
@JoinTable(
    name = "pedido_item",
    joinColumns = @JoinColumn(name = "pedido_id"),
    inverseJoinColumns = @JoinColumn(name = "item_id")
)
private List<ItemCardapio> itensSelecionados = new ArrayList<ItemCardapio>();
```

4.4 Regras de negócio dentro do domínio

Além dos campos, coloquei alguns métodos nas entidades para elas se protegerem sozinhas. A Lanchonete tem métodos para cadastrar cliente, adicionar item ao cardápio e adicionar pedido. Todos recusam valor nulo lançando IllegalArgumentException e já acertam os dois lados da ligação de uma vez, então não corro o risco de ficar com um objeto ligado pela metade.

Código 5 – Regra de domínio na classe Lanchonete

```
public void cadastrarCliente(Cliente cliente) {
    if (cliente == null) {
        throw new IllegalArgumentException("O Cliente não pode ser nulo");
    }
    clientesCadastrados.add(cliente);
    cliente.setLanchonete(this);
}
```

Outro cuidado foi com o encapsulamento das listas. Os getters devolvem Collections.unmodifiableList, para ninguém sair incluindo ou removendo item por fora dos métodos da classe. Só que esse detalhe me deu trabalho na hora dos

testes: quando fui alterar um pedido pelo PUT, o service copiava justamente a lista devolvida pelo getter, e o Hibernate recebia uma lista que ele não conseguia modificar. Resultado, UnsupportedOperationException e erro 500 na cara. Resolvi fazendo uma cópia defensiva dentro do setter, assim o encapsulamento continua de pé e o JPA recebe uma lista que ele pode gerenciar.

Código 6 – Encapsulamento da lista com cópia defensiva no setter

```
public List<ItemCardapio> getItensSelecionados() {  
    return Collections.unmodifiableList(itensSelecionados);  
}  
  
public void setItensSelecionados(List<ItemCardapio> itensSelecionados) {  
    this.itensSelecionados = new ArrayList<>(itensSelecionados);  
}
```

4.5 Controle da resposta em JSON

Como a lanchonete conhece os filhos e os filhos conhecem a lanchonete, se eu jogasse tudo direto no JSON ia entrar em loop: a lanchonete traria os clientes, cada cliente traria a lanchonete de novo, e isso não terminava nunca. Para cortar esse ciclo usei @JsonManagedReference no lado que aparece na resposta e @JsonBackReference no lado que fica escondido.

Código 7 – Os dois lados da mesma ligação

```
// Em Lanchonete: este lado aparece no JSON  
@JsonManagedReference("lanchonete-clientes")  
@OneToMany(mappedBy = "lanchonete")  
private List<Cliente> clientesCadastrados = new ArrayList<Cliente>();  
  
// Em Cliente: este lado é omitido na resposta  
@JsonBackReference("lanchonete-clientes")  
@ManyToOne  
@JoinColumn(name = "lanchonete_id")  
private Lanchonete lanchonete;
```

Na prática, quando eu consulto uma lanchonete a resposta vem com as listas de clientes, de itens do cardápio e de pedidos. Quando consulto um cliente, a lanchonete não aparece repetida lá dentro. Aquele nome entre aspas, o lanchonete-clientes, é para o Jackson saber qual lado combina com qual, já que saem três ligações diferentes da mesma classe. Se um dia eu quiser mostrar também um resumo da lanchonete dentro do cliente, o caminho certo é criar uma classe de resposta menor, com id, nome e CNPJ, em vez de devolver a entidade inteira e cair no loop de novo.

5 VALIDAÇÃO DOS DADOS

5.1 Anotações de Bean Validation usadas no projeto

A validação acontece antes de qualquer regra rodar. As anotações ficam nos campos das entidades e eu escrevi uma mensagem em português para cada uma, porque acho ruim receber uma resposta de erro que não diz o que fazer. O quadro a seguir mostra o que usei e onde.

Quadro 3 – Anotações de validação usadas e onde aparecem

Anotação	O que exige	Onde usei no projeto
@NotBlank	Texto preenchido, e não aceita só espaço em branco.	Nome da lanchonete, do cliente e do item; CPF; CNPJ; ingredientes do lanche.
@NotNull	Valor obrigatório, valendo também para número, data e objeto.	Preço, disponibilidade, lanchonete do item, cliente do pedido e número do pedido.
@NotEmpty	Lista com pelo menos um elemento.	Lista de itens do pedido, para não gravar pedido vazio.
@Size(max = ...)	Tamanho máximo do texto.	Nomes até 150 caracteres e ingredientes até 500.
@Positive	Número maior que zero.	Preço do item e número do pedido.
@Max(value = 5000)	Valor máximo permitido.	Volume da bebida, que trava em 5000 ml.
@PastOrPresent	Data igual ou anterior a agora.	Data e hora de emissão do pedido, que não pode estar no futuro.
@CPF	CPF com dígitos verificadores válidos.	Documento do cliente.
@CNPJ	CNPJ com dígitos verificadores válidos.	Documento da lanchonete.

5.2 O papel do @Valid no controller

Só que essas anotações sozinhas não fazem nada. Quem manda o Spring conferir o objeto é o @Valid, que eu coloquei do lado do @RequestBody no método do controller. Com ele, a validação roda antes da primeira linha do método. Se algum campo estiver errado, o método nem chega a executar: o Spring lança MethodArgumentNotValidException, o tratamento global pega essa exceção e devolve um 400 já explicando o problema.

Código 8 – O @Valid aplicado ao corpo da requisição

```
@PostMapping
public ResponseEntity<Lanche> incluir(@Valid @RequestBody Lanche lanche) {
    lancheService.incluir(lanche);
    URI location = ServletUriComponentsBuilder.fromCurrentRequest()
        .path("/{id}")
        .buildAndExpand(lanche.getId())
        .toUri();
    return ResponseEntity.created(location).body(lanche);
}
```

Coloquei o @Valid no POST e no PUT, que são os que mandam o objeto inteiro. No PATCH deixei ele de fora de propósito: como a alteração parcial manda só alguns campos, exigir o objeto completo faria toda chamada parcial quebrar. Nesse caso, quem decide o que pode ser alterado é o service.

5.3 Validação de regra própria no service

Nem toda regra cabe numa anotação. A busca por nome, por exemplo, não pode aceitar termo vazio nem com uma letra só, senão devolve praticamente o banco inteiro. Essa conferência eu coloquei numa classe utilitária chamada Validation, que os services chamam antes de bater no repositório.

Código 9 – Validação do termo de busca

```
public static void validarTermo(String termo) {
    if (termo == null || termo.isBlank()) {
        throw new IllegalArgumentException(
            "O termo de busca não pode ser vazio ou nulo.");
    }
    if (termo.trim().length() < 3) {
        throw new IllegalArgumentException(
            "O termo de busca deve ter pelo menos 3 caracteres.");
    }
}
```

A exceção que sai daqui também vira 400, porque o tratamento global entende IllegalArgumentException como erro de quem chamou a API, e não como falha da aplicação. A Figura 9, mais para frente, mostra exatamente esse caso acontecendo no Swagger.

6 CAMADA DE CONTROLLER

O controller é a porta de entrada. Ele recebe a requisição, passa os dados para o service e devolve a resposta com o código certo. Fiz questão de não escrever regra de negócio dentro dele, porque senão o controller vira um depósito de tudo e fica impossível de manter.

6.1 Anotações usadas nos controllers

Quadro 4 – Anotações do controller e para que servem

Anotação	Para que serve
@RestController	Marca a classe como controller REST. Já vem com o @ResponseBody embutido, ou seja, o que o método devolve vira JSON direto, sem passar por página nenhuma.
@RequestMapping("/lanches")	Define o caminho base da classe. Todas as rotas daquele controller começam por ele.
@PostMapping	Atende o POST, que é o que usa para criar registro novo.
@GetMapping	Atende o GET, usado para consultar. Aceita um complemento de caminho, como /disponiveis ou /busca.
@PutMapping("/{id}")	Atende o PUT, usado para trocar todos os dados de um registro.
@PatchMapping("/{id}")	Atende o PATCH, usado para mexer só em alguns campos.
@DeleteMapping("/{id}")	Atende o DELETE, usado para excluir.
@PathVariable	Pega um valor que está no próprio endereço, como o id em /lanches/3.
@RequestParam	Pega um valor que vem depois do ponto de interrogação, como o nome em /lanches/busca?nome=burger.
@RequestBody	Transforma o JSON enviado no corpo da requisição em um objeto Java.
@Valid	Manda validar esse objeto antes de o método rodar.
@Operation e @Parameter	São do OpenAPI. É o que escreve no Swagger o resumo da rota, a descrição e o significado de cada parâmetro.

6.2 Exemplo comentado das rotas

A classe começa declarando o caminho base e recebendo o service pelo construtor. Prefiro esse jeito de injetar dependência porque deixa o campo final e deixa claro do que a classe precisa para existir.

Código 10 – Declaração do controller e injeção do service

```

@RestController
@RequestMapping("/lanches")
public class LancheController {

    private final LancheService lancheService;

    public LancheController(LancheService lancheService) {
        this.lancheService = lancheService;
    }
}

```

As duas consultas abaixo mostram bem a diferença entre pegar um valor do caminho e pegar um valor da parte de consulta do endereço. Na primeira, o id vem dentro do endereço e quem captura é o `@PathVariable`. Na segunda, o termo da busca vem depois do ponto de interrogação, e aí quem captura é o `@RequestParam`.

Código 11 – `@PathVariable` e `@RequestParam`

```

@GetMapping("/{id}")
public ResponseEntity<Lanche> getById(
    @Parameter(description = "ID do lanche") @PathVariable Long id) {
    Lanche lanche = lancheService.getById(id);
    return ResponseEntity.ok(lanche);
}

@GetMapping("/busca")
public ResponseEntity<List<Lanche>> getByName(
    @Parameter(description = "Trecho do nome") @RequestParam String nome) {
    List<Lanche> lanches = lancheService.buscarPorNome(nome);
    return ResponseEntity.ok(lanches);
}

```

A busca só ganhou o caminho próprio `/busca` por causa de um problema que apareceu na hora de testar. Nos controllers de cliente e de lanchonete, a listagem e a busca usavam o mesmo caminho e o mesmo GET, mudando só o parâmetro. A aplicação funcionava normal, mas o Swagger mostrava apenas uma das duas, porque no OpenAPI a identidade da rota é o método mais o caminho. Separei as duas e todas as rotas passaram a aparecer e a poder ser testadas.

Nas alterações dá para ver a diferença entre PUT e PATCH já no controller. No PUT eu forço o id que veio no endereço dentro do objeto, senão alguém poderia mandar um id diferente no corpo e acabar alterando outro registro.

```

@PutMapping("/{id}")
public ResponseEntity<Lanche> alterar(@PathVariable Long id,
                                     @Valid @RequestBody Lanche lanche) {
    lanche.setId(id);
    lancheService.alterar(id, lanche);
    return ResponseEntity.ok(lanche);
}

@PatchMapping("/{id}")
public ResponseEntity<Lanche> alterarParcialmente(@PathVariable Long id,
                                                  @RequestBody Lanche lanche) {
    Lanche
    atualizado = lancheService.alterarParcialmente(id, lanche);
    return ResponseEntity.ok(atualizado);
}

@DeleteMapping("/{id}")
public ResponseEntity<Void> deletar(@PathVariable Long id) {
    lancheService.excluir(id);
    return ResponseEntity.noContent().build();
}

```

Código 12 – PUT, PATCH e DELETE

6.3 Códigos HTTP devolvidos em cada operação

Usei `ResponseEntity` como tipo de retorno justamente porque com ele eu controlo o código e os cabeçalhos da resposta, e não só o corpo. O quadro abaixo mostra o que cada operação devolve e o motivo.

Quadro 5 – Códigos de sucesso devolvidos pela API

Operação	Código	Método usado	Por que esse código
POST	201 Created	<code>ResponseEntity.created(location).body(...)</code>	Criei um recurso novo. O cabeçalho <code>Location</code> leva o endereço do registro recém-criado, montado com o id que o banco gerou.
GET	200 OK	<code>ResponseEntity.ok(...)</code>	A consulta deu certo e o corpo traz o resultado. Lista vazia continua sendo 200, porque a consulta funcionou, só não achou nada.
PUT	200 OK	<code>ResponseEntity.ok(...)</code>	Troquei o registro por completo e devolvo no corpo como ele ficou.
PATCH	200 OK	<code>ResponseEntity.ok(...)</code>	Alterei só os campos enviados e devolvo o registro atualizado.

DELETE	204 No Content	ResponseEntity.noContent().build()	A exclusão deu certo e não tem nada para devolver no corpo.
--------	----------------	------------------------------------	---

7 CAMADA DE SERVICE E O CRUD

O service fica no meio do caminho, entre o controller e o repository. É nele que estão as operações do CRUD e as decisões da aplicação, como o que fazer quando o registro pedido não existe. A classe leva `@Service`, que é o que faz o Spring criar e cuidar dessa instância, e recebe o repository pelo construtor, do mesmo jeito que o controller recebe o service.

```
@Service
public class LancheService {

    private final LancheRepository lancheRepository;
    LancheService(LancheRepository lancheRepository) { this.lancheRepository =
        lancheRepository;
    }

    public void incluir(Lanche lanche) { lancheRepository.save(lanche);
    }

    // Create
    public List<Lanche> obterLista() { return lancheRepository.findAll();
    } // Read (lista)

    public Lanche getById(Long id) { // Read (um registro)
        return lancheRepository.findById(id).orElseThrow(() -> new
            RecursoNaoEncontradoException(
                "Nenhum objeto encontrado para o id " + id + "."));
    }

    public void excluir(Long id) { // Delete Lanche lanche = getById(id);
        lancheRepository.delete(lanche);
    }
}
```

Código 13 – Criar, ler, atualizar e excluir

O `getById` é o método que mais me poupou trabalho no projeto todo. O `findById` do Spring Data devolve um `Optional`, que pode ou não ter o registro dentro. Em vez de espalhar verificação de nulo por todo lado, uso o `orElseThrow` para lançar a `RecursoNaoEncontradoException` quando não acha nada. Como alterar e excluir também chamam o `getById` antes de qualquer coisa, qualquer id que não existe cai num 404 padronizado, em qualquer rota, sem eu precisar repetir código nenhum.

7.1 A diferença entre alterar por completo e alterar em parte

O método que o PUT chama copia todos os campos que vieram para o registro que já existe. Se algum campo não vier, ele é apagado mesmo, porque a ideia do PUT é substituir o recurso inteiro. Já o método do PATCH confere campo por campo e só copia o que veio preenchido.

Código 14 – Alteração completa (PUT) e alteração parcial (PATCH)

```
public void alterar(Long id, Lanche lanche) {
    Lanche existente = getById(id);
    existente.setNome(lanche.getNome());
    existente.setPreco(lanche.getPreco());
    existente.setDisponivel(lanche.getDisponivel());
    existente.setIngredientes(lanche.getIngredientes());
    existente.setArtesanal(lanche.isArtesanal());
    lancheRepository.save(existente);
}

public Lanche alterarParcialmente(Long id, Lanche lanche) {
    Lanche existente = getById(id);
    aplicarAlteracoesParciais(existente, lanche);
    return lancheRepository.save(existente);
}

private void aplicarAlteracoesParciais(Lanche existente, Lanche novosDados) {
    if (novosDados.getNome() != null) {
        existente.setNome(novosDados.getNome());
    }
    if (novosDados.getPreco() != null) {
        existente.setPreco(novosDados.getPreco());
    }
    // e assim por diante para os outros campos
}
```

Nos dois casos eu busco o registro no banco antes de alterar. Isso garante que quem vai ser salvo é o objeto que o JPA já conhece, e não um objeto solto criado a partir do JSON, que acabaria virando um registro novo em vez de atualizar o antigo.

8 CAMADA DE REPOSITORY

O repository é só uma interface, e essa foi a parte que mais me surpreendeu. Não escrevi nenhuma implementação: herdando de JpaRepository, a interface já vem com save, findAll, findById, delete e um monte de outros métodos prontos, e o Spring Data cria a classe concreta quando a aplicação sobe.

Os dois tipos entre os sinais de menor e maior são a entidade e o tipo da chave primária.

Código 15 – Repository com consultas derivadas do nome do método

```
public interface LancheRepository extends JpaRepository<Lanche, Long> {  
    List<Lanche> findByDisponivelTrue();  
    List<Lanche> findByNameContainingIgnoreCase(String nome);  
}
```

Os dois métodos que declarei aproveitam a consulta derivada. O Spring Data lê o nome do método e monta o SQL sozinho: `findByDisponivelTrue` traz só os itens com o campo `disponivel` verdadeiro, e `findByNameContainingIgnoreCase` traz os itens cujo nome contém o trecho que eu passei, sem diferenciar maiúscula de minúscula. No fim é o mesmo que um `LIKE`, mas sem eu escrever SQL na mão.

9 TRATAMENTO GLOBAL DE EXCEÇÕES E CÓDIGOS DE ERRO

Se eu não tratasse nada, qualquer erro chegaria no usuário como um 500 com a pilha de erro do Java, que ninguém entende e que ainda expõe detalhe interno da aplicação. Para evitar isso, juntei o tratamento numa classe só, marcada com `@RestControllerAdvice`. Essa anotação faz a classe valer para todos os controllers ao mesmo tempo, então não precisei encher os métodos de `try-catch`.

9.1 A classe `GlobalExceptionHandler`

Dentro dela, cada método marcado com `@ExceptionHandler` cuida de um tipo de exceção, e todos chamam o mesmo método privado `criarResposta`, que monta o corpo padrão. Com isso, qualquer erro da API sai sempre no mesmo formato.

Código 16 – Estrutura do tratamento global de erros

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    private ResponseEntity<ErroResponse> criarResposta(HttpStatus status,
                                                         String mensagem) {
        ErroResponse erro = new ErroResponse(status.value(),
                                              status.getReasonPhrase(),
                                              mensagem,
                                              LocalDateTime.now());
        return ResponseEntity.status(status).body(erro);
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ErroResponse> tratarErroValidacao(
        MethodArgumentNotValidException exception) {
        String mensagem = exception.getBindingResult().getFieldErrors().stream()
            .map(erro -> erro.getField() + ": " + erro.getDefaultMessage())
            .collect(Collectors.joining("; "));
        return criarResposta(HttpStatus.BAD_REQUEST, mensagem);
    }

    @ExceptionHandler(RecursoNaoEncontradoException.class)
    public ResponseEntity<ErroResponse> tratarRecursoNaoEncontrado(
        RecursoNaoEncontradoException exception) {
        return criarResposta(HttpStatus.NOT_FOUND, exception.getMessage());
    }
}
```

O tratamento do erro de validação é o que eu acho mais interessante. O `MethodArgumentNotValidException` carrega a lista de todos os campos que foram reprovados. O código percorre essa lista, junta o nome do campo com a mensagem que eu escrevi na anotação e separa tudo por ponto e vírgula. Por isso a resposta não fica só num genérico dizendo que deu erro: ela diz qual campo está errado e o que se esperava.

9.2 O corpo padrão do erro

A classe `ErroResponse` é quem define o formato devolvido em toda falha. São quatro informações: o número do status, o nome do erro, a mensagem contando o motivo e a data e hora em que aconteceu. Um retorno real, que capturei no Swagger, fica assim:

Código 17 – Resposta de erro devolvida pela API

```
{
  "status": 400,
  "erro": "Bad Request",
  "mensagem": "O termo de busca deve ter pelo menos 3 caracteres.",
  "dataHora": "2026-08-29T14:49:51.3678177"
}
```

9.3 Quais erros acontecem e por quê

O quadro abaixo reúne os erros que a API pode devolver. Os quatro primeiros passam pelo `GlobalExceptionHandler` e saem no formato que mostrei acima. Os três últimos o próprio Spring resolve antes de a requisição chegar no meu código, então eles saem no formato padrão do framework mesmo.

Quadro 6 – Códigos de erro, origem e motivo

Status	Origem	Quando acontece
400 Bad Request	<code>MethodArgumentNotValidException</code> (tratada)	Algum campo não passou no Bean Validation: nome vazio, preço zero ou negativo, CPF ou CNPJ inválido, pedido sem item ou data de emissão no futuro.
400 Bad Request	<code>IllegalArgumentException</code> (tratada)	O argumento não faz sentido para a regra, como buscar com termo vazio ou com menos de três caracteres.
404 Not Found	<code>RecursoNaoEncontradoException</code> (tratada)	O id informado não existe no banco. Acontece ao consultar, alterar ou excluir um registro que não está lá, já que todos passam pelo <code>getById</code> .
409 Conflict	<code>IdentificadorDuplicadoException</code> (tratada)	O cadastro bate de frente com um identificador que já existe.
405 Method Not Allowed	Spring MVC	O caminho existe, mas não aceita aquele método. Por exemplo, mandar DELETE para <code>/lanches</code> , que só aceita DELETE com o id.
415 Unsupported Media Type	Spring MVC	O corpo veio num formato diferente do esperado, normalmente sem o cabeçalho <code>Content-Type</code> igual a <code>application/json</code> .
500 Internal Server Error	Falha não prevista	Erro interno que não tem tratamento específico. Foi o que aconteceu no PUT de pedido antes de eu corrigir a lista de itens, que contei na seção 4.4.

Uma coisa que ficou clara para mim testando isso tudo é a diferença entre as duas famílias de erro. Código que começa com 4 quer dizer que o problema está no que foi enviado, e quem chamou a API consegue resolver sozinho lendo a mensagem. Código que começa com 5 é culpa da aplicação: quem chamou não tem o que fazer, e sobra para mim ir olhar o log do servidor.

10 DOCUMENTAÇÃO COM SWAGGER E RESULTADOS DOS TESTES

Quem gera a documentação é o Springdoc OpenAPI, e ela abre no Swagger UI, no endereço local da aplicação. Cada rota mostra o método, o caminho, o resumo que escrevi no `@Operation`, a descrição de cada parâmetro do `@Parameter` e um exemplo do corpo esperado. O botão Execute dispara a requisição de verdade e mostra o código e o corpo da resposta, então consegui conferir a API inteira sem precisar de outro programa.

No total rodei 44 operações, passando pelos seis controllers e por todos os tipos de chamada. Os cadastros voltaram 201 Created, as consultas, buscas e alterações voltaram 200 OK e as exclusões voltaram 204 No Content. Fora essa conferência manual, escrevi seis testes de integração para os pontos que deram problema, fazendo questão de reproduzir a falha primeiro e só depois corrigir, para ter certeza de que o teste estava pegando a coisa certa.

10.1 Quadro geral das rotas verificadas

Quadro 7 – Rotas executadas no Swagger UI e código HTTP obtido

Controller	Método	Rota	Resultado
Lanchonete	POST	/lanchonetes	201
Lanchonete	GET	/lanchonetes	200
Lanchonete	GET	/lanchonetes/{id}	200
Lanchonete	GET	/lanchonetes/busca	200
Lanchonete	PUT	/lanchonetes/{id}	200
Lanchonete	PATCH	/lanchonetes/{id}	200
Lanchonete	DELETE	/lanchonetes/{id}	204
Cliente	POST	/clientes	201
Cliente	GET	/clientes	200
Cliente	GET	/clientes/{id}	200
Cliente	GET	/clientes/busca	200
Cliente	PUT	/clientes/{id}	200
Cliente	PATCH	/clientes/{id}	200
Cliente	DELETE	/clientes/{id}	204

Controller	Método	Rota	Resultado
Lanche	POST	/lanches	201
Lanche	GET	/lanches	200
Lanche	GET	/lanches/{id}	200
Lanche	GET	/lanches/disponiveis	200
Lanche	GET	/lanches/busca	200
Lanche	PUT	/lanches/{id}	200
Lanche	PATCH	/lanches/{id}	200
Lanche	DELETE	/lanches/{id}	204
Bebida	POST	/bebidas	201
Bebida	GET	/bebidas	200
Bebida	GET	/bebidas/{id}	200
Bebida	GET	/bebidas/disponiveis	200
Bebida	GET	/bebidas/busca	200
Bebida	PUT	/bebidas/{id}	200
Bebida	PATCH	/bebidas/{id}	200
Bebida	DELETE	/bebidas/{id}	204
ItemCardapio	POST	/itemcardapios	201
ItemCardapio	GET	/itemcardapios	200
ItemCardapio	GET	/itemcardapios/{id}	200
ItemCardapio	GET	/itemcardapios/disponiveis	200
ItemCardapio	GET	/itemcardapios/busca	200
ItemCardapio	PUT	/itemcardapios/{id}	200
ItemCardapio	PATCH	/itemcardapios/{id}	200
ItemCardapio	DELETE	/itemcardapios/{id}	204
Pedido	POST	/pedidos	201
Pedido	GET	/pedidos	200
Pedido	GET	/pedidos/{id}	200
Pedido	PUT	/pedidos/{id}	200
Pedido	PATCH	/pedidos/{id}	200
Pedido	DELETE	/pedidos/{id}	204

10.2 Evidências selecionadas

Não faz sentido colocar aqui as 44 telas, que são bem parecidas entre si. Escolhi as capturas que representam os seis controllers e os principais tipos de operação, mais um exemplo de erro. As outras ficaram salvas na pasta docs/screenshots/swagger do repositório.

Figura 1 – Cadastro de lanchonete com resposta 201 Created

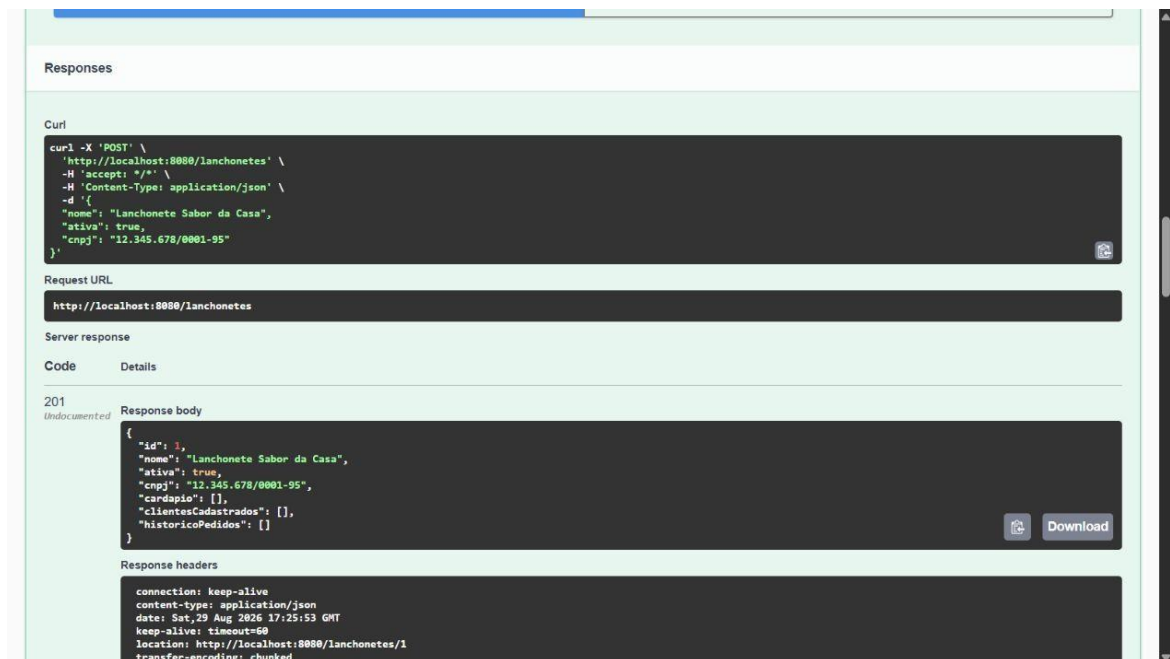


Figura 2 – Cadastro de cliente com resposta 201 Created



Figura 3 – Cadastro de lanche com resposta 201 Created

Curl

```
curl -X 'POST' \
  'http://localhost:8080/lanches' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "tipo": "lanche",
    "nome": "X-Burger Artesanal",
    "preco": 32.5,
    "disponivel": true,
    "lanchonete": {
      "id": 1
    },
    "ingredientes": "Pão, carne, queijo e salada",
    "artesanal": true
  }'
```

Request URL

http://localhost:8080/lanches

Server response

Code Details

201
Undocumented

Response body

```
{
  "tipo": "lanche",
  "id": 1,
  "nome": "X-Burger Artesanal",
  "preco": 32.5,
  "disponivel": true,
  "ingredientes": "Pão, carne, queijo e salada"
}
```

Response headers

```
connection: keep-alive
content-type: application/json
date: Sat, 29 Aug 2026 17:38:48 GMT
keep-alive: timeout=60
location: http://localhost:8080/lanches/1
transfer-encoding: chunked
```

Figura 4 – Cadastro de bebida com resposta 201 Created

Curl

```
curl -X 'POST' \
  'http://localhost:8080/bebidas' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "tipo": "bebida",
    "nome": "Suco de Laranja",
    "preco": 9.9,
    "disponivel": true,
    "lanchonete": {
      "id": 1
    },
    "volumeML": 500,
    "alcoolica": false
  }'
```

Request URL

http://localhost:8080/bebidas

Server response

Code Details

201
Undocumented

Response body

```
{
  "tipo": "bebida",
  "id": 3,
  "nome": "Suco de Laranja",
  "preco": 9.9,
  "disponivel": true,
  "volumeML": 500
}
```

Response headers

```
connection: keep-alive
content-type: application/json
date: Sat, 29 Aug 2026 17:33:51 GMT
keep-alive: timeout=60
location: http://localhost:8080/bebidas/3
transfer-encoding: chunked
```

Figura 5 – Consulta geral dos itens do cardápio com resposta 200 OK

The screenshot shows a REST client interface with a blue header bar containing 'Execute' and 'Clear' buttons. Below the header, the 'Responses' section is active. The 'Curl' field contains the command: `curl -X 'GET' \ 'http://localhost:8080/itemcardapios' \ -H 'accept: */*'` . The 'Request URL' field shows `http://localhost:8080/itemcardapios`. The 'Server response' section displays a '200' status code and a 'Response body' containing a JSON array of two items: a burger and a juice. The interface has a light blue background.

```
curl -X 'GET' \
'http://localhost:8080/itemcardapios' \
-H 'accept: */*'

Request URL
http://localhost:8080/itemcardapios

Server response
Code    Details
200     Response body
{
  {
    "tipo": "lanche",
    "id": 1,
    "nome": "X-Burger Especial",
    "preco": 33.9,
    "disponivel": true,
    "ingredientes": "Pão, carne, queijo, bacon e salada"
  },
  {
    "tipo": "bebida",
    "id": 3,
    "nome": "Suco de Laranja Natural",
    "preco": 9.5,
    "disponivel": true,
    "volumeML": 500
  },
}
```

Figura 6 – Cadastro de pedido com resposta 201 Created

The screenshot shows a REST client interface with a light green header bar. The 'Request URL' field contains `http://localhost:8080/pedidos`. The 'Server response' section displays a '201' status code with the label 'Undocumented' and a 'Response body' containing a JSON object representing a newly created order. The interface has a light green background.

```
"dataHoraEmissao": "2026-08-29T13:00:00",
"ativo": true,
"cliente": {
  "id": 1
},
"itensSelecioneados": [
  {
    "tipo": "lanche",
    "id": 1
  },
  {
    "tipo": "bebida",
    "id": 3
  }
],
"lancheonete": {
  "id": 1
}
}

Request URL
http://localhost:8080/pedidos

Server response
Code    Details
201     Undocumented Response body
{
  "id": 1,
  "numeroPedido": 1001,
  "dataHoraEmissao": "2026-08-29T13:00:00",
  "ativo": true,
  "cliente": {
    "id": 1,
    "nome": null,
    "cpf": null
  },
  "itensSelecioneados": [
    {
      "tipo": "lanche",
      "id": 1,
      "nome": null,
      "preco": null,
      "disponivel": null,
      "ingredientes": null
    }
  ]
}
```

Figura 7 – Alteração completa de pedido com resposta 200 OK

The screenshot displays a REST client interface with a dark theme. The top section shows a JSON request body for a PUT operation. Below it, the 'Request URL' is set to `http://localhost:8080/pedidos/1`. The 'Server response' section shows a 200 status code. The 'Response body' is a JSON object representing the updated order.

```
Request body: {
  "dataHoraEmissao": "2026-08-29T13:15:00",
  "ativo": true,
  "cliente": {
    "id": 1
  },
  "itensSelecioneados": [
    {
      "tipo": "lanche",
      "id": 1
    },
    {
      "tipo": "bebida",
      "id": 2
    }
  ],
  "lancheonete": {
    "id": 1
  }
}
```

Request URL: `http://localhost:8080/pedidos/1`

Server response: 200

Response body:

```
{
  "id": 1,
  "numeroPedido": 1001,
  "dataHoraEmissao": "2026-08-29T13:15:00",
  "ativo": true,
  "cliente": {
    "id": 1,
    "nome": null,
    "cpf": null
  },
  "itensSelecioneados": [
    {
      "tipo": "lanche",
      "id": 1,
      "nome": null,
      "preco": null,
      "disponivel": null,
      "ingredientes": null
    }
  ]
}
```

Figura 8 – Alteração parcial de pedido com resposta 200 OK

The screenshot displays a REST client interface with a light green theme. The top section shows a JSON request body for a PATCH operation. Below it, the 'Request URL' is set to `http://localhost:8080/pedidos/1`. The 'Server response' section shows a 200 status code. The 'Response body' is a JSON object representing the partially updated order.

```
Request body: {
  "ativo": false,
  "itensSelecioneados": [
    {
      "tipo": "lanche",
      "id": 1
    },
    {
      "tipo": "bebida",
      "id": 2
    }
  ]
}
```

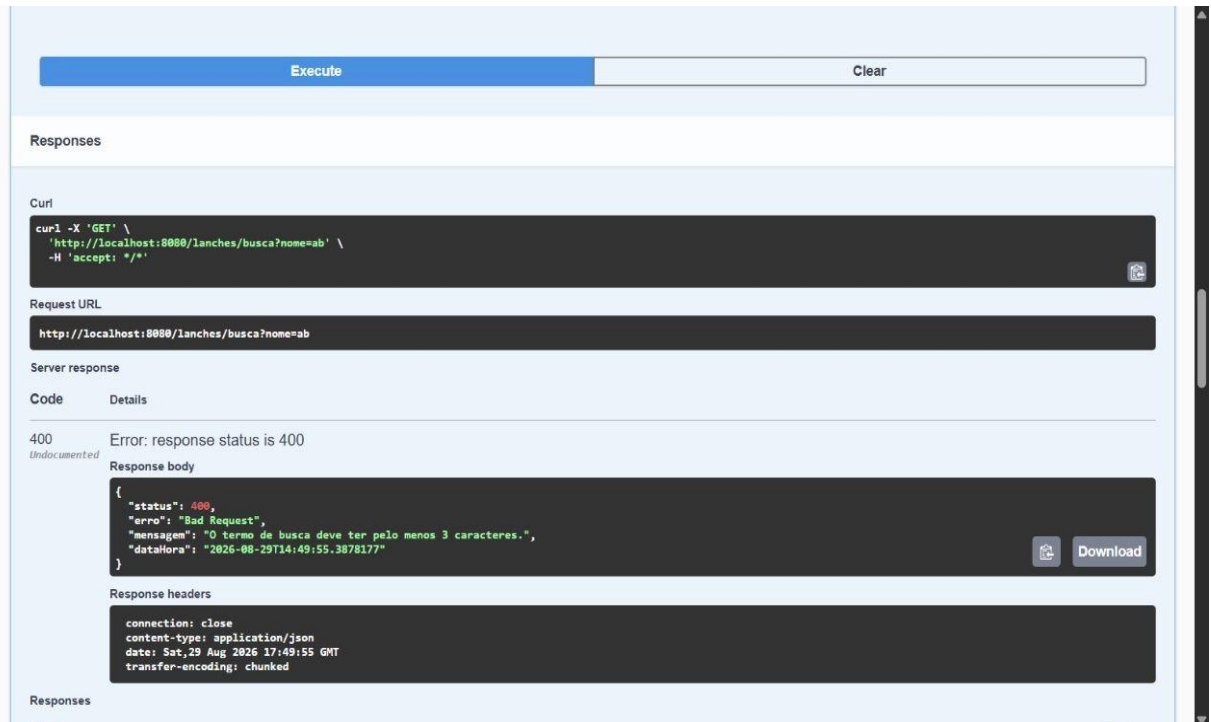
Request URL: `http://localhost:8080/pedidos/1`

Server response: 200

Response body:

```
{
  "id": 1,
  "numeroPedido": 1001,
  "dataHoraEmissao": "2026-08-29T13:15:00",
  "ativo": false,
  "cliente": {
    "id": 1,
    "nome": "Carlos Silva",
    "cpf": "529.982.247-25"
  },
  "itensSelecioneados": [
    {
      "tipo": "lanche",
      "id": 1,
      "nome": "X-Burger Artesanal",
      "preco": 32.5,
      "disponivel": true,
      "ingredientes": "Pão, carne, queijo e salada"
    }
  ]
}
```

Figura 9 – Erro de validação com resposta 400 Bad Request



11 CONSIDERAÇÕES FINAIS

No fim das contas a API ficou do jeito que eu queria: ela grava as entidades, respeita os relacionamentos, valida o que recebe e faz cadastro, consulta, busca, alteração total, alteração parcial e exclusão, sempre devolvendo o código HTTP adequado. O Swagger ajudou muito nessa reta final, porque mostra no mesmo lugar a documentação e a resposta real de cada chamada.

A parte que mais valeu a pena foi justamente tropeçar nos problemas. A separação das rotas de busca, o campo tipo que o cadastro de item pedia e a lista imutável que derrubava o PUT de pedido são coisas que eu não teria percebido só relendo o código. Só apareceram quando rodei o fluxo inteiro, da requisição até o banco. Ficou a lição de que testar a aplicação de pé é tão importante quanto escrevê-la.

Se eu fosse continuar o projeto, o próximo passo seria criar classes de resposta reduzidas, para conseguir mostrar um resumo da lanchonete dentro dos objetos filhos sem cair no loop do JSON, e depois colocar paginação nas listagens e um banco que guarde os dados entre uma execução e outra.

Por último, registro que usei inteligência artificial como apoio em pontos específicos: revisar a documentação, ajudar a achar a causa dos erros que apareceram nos testes e organizar este relatório. As decisões sobre o modelo, a implementação e a conferência final foram minhas.